Table des matières

Cours 03 — Monter le projet dbt et le STAGING ..... 1
Objectifs ..... 2
1. Ce qu'est un modèle, et ce que dbt ajoute ..... 2
2. Le projet et sa configuration ..... 3
3. La connexion, et pourquoi elle vit ailleurs ..... 3
4. Les sources ..... 4
5. Où dbt écrit ce qu'il produit ..... 4
6. Les quatre gestes du `STAGING` ..... 5
7. L'état courant, et ce qu'il répare ..... 6
8. Les matérialisations ..... 6
9. `ref()` et le graphe ..... 7
10. Sur une autre plateforme ..... 7
Points clés à retenir ..... 8
Exercices ..... 8

# Cours 03 — Monter le projet dbt et le STAGING

*Théorie · Cours 3/9 — Formation Analytics Engineering · intermédiaire*

Où vous êtes : `RAW` → **`STAGING`** → `INTERMEDIATE` → `MARTS`. Le Cours 03 ouvre l'outil du métier et bâtit la première couche, celle qui rend chaque source lisible sans encore rien joindre.

En 30 secondes :

- Un modèle dbt est un fichier qui contient un `SELECT`, et rien d'autre. dbt l'enveloppe, lui donne un nom, décide s'il devient une vue ou une table, et l'exécute dans le bon ordre.
- Ce que dbt apporte au SQL n'est pas de la puissance : c'est un dépôt Git, un graphe de dépendances, des tests et une documentation qui sortent du code lui-même.
- Une source se déclare avant de se lire. La fonction `source()` remplace le nom en dur, et c'est ce qui permet à dbt de savoir d'où vient chaque chose.



© 2026 iAlexMG. Tous droits réservés.

- Le `STAGING` fait quatre gestes et un seul de plus : renommer, typer, décoder, et tirer l'état courant. Il ne joint rien.
- `ref()` est la clé de voûte : elle relie deux modèles, et c'est de ces liens que dbt tire l'ordre d'exécution, le graphe et la documentation.

L'idée du cours. Vous savez déjà écrire les requêtes de cette couche : vous les avez écrites au labo précédent, à la main. Ce cours ne vous apprend pas un meilleur SQL, il vous apprend à le tenir : un fichier par modèle, un dépôt, un graphe qui sait dans quel ordre exécuter, et un outil qui refuse de deviner ce que vous n'avez pas déclaré. Au labo, vous monterez le projet et écrirez vos premières vues de staging, dont l'une retrouvera les 25 483 commandes de Boréal malgré le lot livré deux fois.

## Objectifs

- Dire ce qu'est un modèle dbt, et ce que dbt ajoute autour d'un `SELECT`.
- Monter un projet dbt dans un dépôt existant, et dire où vit le fichier de connexion et pourquoi.
- Déclarer une source et la lire par `source()`.
- Nommer les quatre gestes du `STAGING`, et le geste qu'il refuse.
- Écrire une vue de staging qui tire l'état courant d'une table livrée par lots.
- Relier deux modèles par `ref()`, et lire le graphe qui en résulte.
- Choisir une matérialisation, et dire ce que chacune coûte.
- Expliquer où dbt écrit ses objets, et comment on le décide.

---

*Ce cours se lit en deux séances. La première va jusqu'à la fin de la section 5 : l'outil, le projet monté, les sources déclarées. La seconde porte les modèles de staging et le graphe.*

---

## 1. Ce qu'est un modèle, et ce que dbt ajoute

Un modèle dbt est un fichier `.sql` qui contient un `SELECT`. Pas de `create table`, pas de `insert`, pas de point-virgule final : juste la requête qui décrit ce que le modèle doit contenir.

Le nom du fichier devient le nom de l'objet dans la base. `stg_concessionnaires.sql` produit `stg_concessionnaires`, et c'est dbt qui écrit le `create` autour, selon la matérialisation que vous avez choisie.



© 2026 iAlexMG. Tous droits réservés.

Ce que l'outil ajoute tient en quatre points, et aucun ne concerne le SQL lui-même.

| Ce que dbt ajoute | Ce que ça change |
|---|---|
| Un fichier par modèle, dans un dépôt Git | une transformation se relit, se compare, se révise |
| Un graphe de dépendances, déduit du code | l'ordre d'exécution n'est plus tenu à la main |
| Des tests déclarés à côté du modèle | une règle métier se vérifie à chaque exécution |
| Une documentation générée depuis le code | elle ne peut pas dériver de ce qu'elle décrit |

Le troisième et le quatrième arrivent au Cours 05. Ce cours-ci pose les deux premiers, et ils suffisent déjà à changer la façon de travailler : le SQL du Labo 02, que vous avez tapé dans une ligne de commande, était juste et il a disparu à la fermeture du terminal.

À retenir — dbt n'est pas un moteur. Il ne calcule rien : il compile votre SQL, l'envoie à DuckDB et lit la réponse. Tout ce qui est lent dans un projet dbt est lent dans la base, et se répare dans la base.

## 2. Le projet et sa configuration

Un projet dbt est un dossier qui contient un `dbt_project.yml`. C'est ce fichier qui fait exister le projet, et dbt refuse de travailler sans lui.

| Ce qu'il déclare | À quoi ça sert |
|---|---|
| `name` | le nom du projet, repris dans les configurations |
| `profile` | quel profil de connexion utiliser |
| `model-paths`, `macro-paths` | où chercher les fichiers |
| le bloc `models:` | les réglages qui s'appliquent par défaut, dossier par dossier |

Le bloc `models:` mérite qu'on s'y arrête, parce qu'il gouverne tout le reste. Les réglages y descendent en cascade, du projet vers le dossier, puis vers le fichier. Un `+materialized: view` posé sur le dossier `staging` s'applique à tous ses modèles ; un modèle qui déclare autre chose dans son propre en-tête l'emporte sur lui.

Cette cascade est ce qui permet de ne rien répéter. Vous ne dites pas quarante fois que vos vues de staging sont des vues : vous le dites une fois, sur le dossier.

*Labo : `Labo_03_AE.md`, étape 2.*

## 3. La connexion, et pourquoi elle vit ailleurs



© 2026 iAlexMG. Tous droits réservés.

Le `dbt_project.yml` ne dit pas comment se connecter à la base. C'est un second fichier qui le dit, `profiles.yml`, et il vit hors du dépôt.

La raison est celle que le Cours 01 a déjà posée pour le `.gitignore` : ce fichier porte de quoi se connecter. Sous DuckDB il ne porte qu'un chemin, ce qui paraît anodin ; les annexes Snowflake et Databricks lui font porter un compte, un utilisateur et de quoi s'authentifier. Une règle qui ne s'applique que lorsqu'elle compte ne s'applique jamais, parce que personne ne se souvient de la rétablir.

dbt le cherche par défaut dans un dossier `.dbt` de votre répertoire personnel. Vous pouvez lui en désigner un autre par la variable d'environnement `DBT_PROFILES_DIR`, ce que fait l'intégration continue au Cours 08.

Piège — un chemin relatif dans `profiles.yml` se résout depuis le dossier où vous lancez la commande, pas depuis le projet. La même commande lancée d'un cran plus haut cherche alors la base ailleurs et échoue sur un fichier introuvable. Écrivez un chemin absolu : le fichier est propre à votre poste de toute façon, puisqu'il ne se versionne pas.

*Labo : `Labo_03_AE.md`, étape 1.*

## 4. Les sources

Une source est une table que vous n'avez pas fabriquée et que vous vous engagez à ne pas modifier : chez Boréal, les 22 tables de la couche `RAW`.

Vous pourriez écrire `from raw.erp_kna1` dans vos modèles, et cela fonctionnerait. Déclarer la source dans un fichier YAML, puis écrire `{{ source('raw', 'erp_kna1') }}`, coûte trois lignes de plus et rapporte quatre choses.

| **Ce qu'on gagne** | **Concrètement** |
|---|---|
| Le graphe remonte jusqu'à la source | on voit ce qui casse si la source change |
| Le nom du schéma vit à un seul endroit | passer d'un environnement à l'autre ne réécrit aucun modèle |
| La documentation les connaît | elles apparaissent dans le lignage, avec leur description |
| La fraîcheur devient vérifiable | dbt peut dire si une source n'a pas été alimentée, au Cours 08 |

La deuxième ligne est celle qu'on mesure le plus vite. Un projet qui écrit le nom du schéma en dur dans quarante modèles ne se déplace pas ; un projet qui passe par `source()` change une ligne de YAML.

*Labo : `Labo_03_AE.md`, étapes 4 et 5.*

## 5. Où dbt écrit ce qu'il produit

dbt écrit dans le schéma que le profil désigne, et il permet de le préciser modèle par modèle. C'est ici qu'il faut se méfier d'un comportement qui surprend tout le monde une fois.



© 2026 iAlexMG. Tous droits réservés.

Par défaut, un schéma déclaré dans le projet ne remplace pas celui du profil : il s'y ajoute. Un profil qui vise `staging` et un dossier qui déclare `+schema: marts` écrivent dans `staging_marts`, et non dans `marts`.

Ce comportement a une raison : sur une plateforme partagée, il évite que deux développeurs écrivent dans le même schéma. Il a aussi un inconvénient, qui est de donner des noms que personne n'a voulus.

La parade tient en une macro que tout projet finit par écrire, `generate_schema_name`, et qui dit à dbt de prendre le nom déclaré tel quel. Le Labo 03 vous la fait écrire, et elle ne changera qu'une fois d'ici le capstone, au Cours 07, quand le projet aura deux environnements à séparer.

*Labo : `Labo_03_AE.md`, étape 3.*

*Renvoi : les vues sont au Cours 03 de SQL, section 7, et le typage à son Cours 02, section 4.*

---

## 6. Les quatre gestes du `STAGING`

*Ici commence la seconde séance de lecture.*

La couche `STAGING` a une responsabilité, et une seule : rendre chaque source lisible. Une vue par table livrée, et quatre gestes.

| **Le geste** | **Ce qu'il fait chez Boréal** |
|---|---|
| Renommer | `KUNNR` devient `code_concessionnaire`, `NETWR` devient `valeur_nette` |
| Typer | la date SAP `20230302` devient une date, le montant en texte devient un décimal |
| Décoder | le code `01` devient Or, d'après le dictionnaire de l'univers |
| Tirer l'état courant | des trois livraisons d'une commande, garder la plus récente |

Le quatrième est le plus important, et c'est celui qu'on oublie. La couche `RAW` garde chaque livraison, par contrat : `raw.erp_vbak` porte 26 538 lignes pour 25 483 commandes. Un modèle de staging qui se contenterait de renommer et de typer transmettrait ces doublons à toute la chaîne, et chaque total serait faux de quelques pour cent, sans que rien ne le signale.

Tirer l'état courant se fait par une fonction de fenêtre : numéroter les livraisons de chaque clé, de la plus récente à la plus ancienne, et ne garder que la première.

*Renvoi : les fonctions de fenêtre sont au Cours 04 de SQL, et les expressions de table communes au Cours 05.*



© 2026 iAlexMG. Tous droits réservés.

Et un geste que le `STAGING` refuse : joindre une autre source. La tentation est constante, parce qu'une vue qui ramène déjà le nom du concessionnaire à côté de la commande est plus agréable. Mais une jointure est une décision de modélisation, et la modélisation a ses couches. Un `STAGING` qui joint devient un endroit où le métier se fait, en dehors de l'endroit prévu pour lui.

À retenir — la règle se dit en une phrase. Au staging, décoder une colonne oui, joindre une autre table non.

*Labo : `Labo_03_AE.md`, étapes 6 et 7.*

## 7. L'état courant, et ce qu'il répare

Tirer l'état courant fait disparaître un problème, et il faut savoir lequel.

Chez Boréal, un concessionnaire a passé une commande avant que sa fiche n'arrive dans les extraits : sa commande est livrée au lot de mai 2024, sa fiche au lot de juin. À l'époque, cette commande pointait un concessionnaire inconnu.

Sur l'état courant, le problème n'existe plus : la fiche est là depuis juin, et toutes les commandes trouvent leur concessionnaire. Un contrôle qui compte les commandes orphelines rend zéro.

C'est ce que vous voulez, la plupart du temps. Mais retenez que ce zéro est le produit d'un choix : vous avez décidé de regarder le référentiel d'aujourd'hui plutôt que celui de l'époque. Le jour où la dimension passera en type 2, au Cours 05, la même question donnera une autre réponse, et les deux seront justes.

*Labo : `Labo_03_AE.md`, étapes 7 et 8.*

## 8. Les matérialisations

Un modèle devient un objet dans la base, et vous choisissez lequel.

| Matérialisation | Ce que dbt écrit | Quand la prendre |
|---|---|---|
| `view` | une vue : la requête est rejouée à chaque lecture | le staging, et tout ce qui est lu rarement |
| `table` | une table : le résultat est écrit une fois | les `MARTS`, lus souvent et par plusieurs |
| `ephemeral` | rien : le SQL est injecté dans les modèles qui l'appellent | un calcul intermédiaire qui ne mérite pas d'objet |
| `incremental` | une table complétée par tranches | un gros fait qui s'allonge, au Cours 07 |

Le staging est en vues, et c'est un choix qui se défend en une phrase : une vue ne coûte rien à construire, elle est toujours à jour, et le staging ne fait que renommer et typer, donc son coût de lecture est négligeable.



© 2026 iAlexMG. Tous droits réservés.

Les deux dernières arrivent plus tard, et l'`ephemeral` mérite un avertissement : un modèle `ephemeral` n'existe pas dans la base, donc il ne s'interroge pas, et une erreur dans son SQL apparaît dans le modèle qui l'appelle. Commode et déroutant.

*Labo : `Labo_03_AE.md`, étapes 6 et 7.*

## 9. `ref()` et le graphe

C'est la pièce qui fait de dbt autre chose qu'un lanceur de scripts.

Quand un modèle en lit un autre, il ne le nomme pas en dur : il écrit `{{ ref('stg_commandes') }}`. dbt remplace cela par le nom réel de l'objet, dans le bon schéma, et surtout il enregistre le lien.

De ces liens, il déduit le graphe, et du graphe il tire trois choses :

- l'ordre d'exécution, qui n'a plus à être tenu à la main ;
- ce qui doit être reconstruit quand un modèle change ;
- la documentation du lignage, qui montre d'où vient chaque table.

Le graphe se regarde en ligne de commande, par `dbt ls`, qui liste les modèles et les sources dans l'ordre où dbt les connaît. Dans un projet de quarante modèles, cette liste est la première chose qu'on consulte quand un chiffre surprend.

Piège — un modèle qui nomme un autre modèle en dur fonctionne, et casse le graphe. dbt ne voit pas le lien, donc il peut exécuter les deux dans le mauvais ordre, et la documentation ment sur le lignage. La règle n'a pas d'exception : `source()` pour ce qui vient d'ailleurs, `ref()` pour tout ce que le projet fabrique.

*Labo : `Labo_03_AE.md`, étapes 5 et 8.*

## 10. Sur une autre plateforme

Ce cours tourne sur DuckDB, et tout ce qu'il enseigne vaut ailleurs : un modèle est un `SELECT`, une source se déclare, `ref()` fait le graphe. Ce qui change d'une plateforme à l'autre tient à trois choses, et l'annexe les traite.

| **Ce qui change** | **Sur Snowflake** | **Sur Databricks** |
|---|---|---|
| L'adaptateur | `dbt-snowflake` | `dbt-databricks` |
| La connexion | compte, utilisateur, rôle, entrepôt virtuel, paire de clés | hôte, chemin de l'entrepôt SQL, OAuth par le navigateur |
| Ce qu'on paie | le temps d'entrepôt allumé | le temps d'entrepôt SQL |



© 2026 iAlexMG. Tous droits réservés.

Un mot sur un terme qui prête à confusion, et qu'il vaut mieux régler une fois pour toutes. Sur Snowflake, une table *transient* est une table sans protection de récupération après suppression : elle coûte moins cher à stocker et se perd définitivement si on l'efface. Deux choses n'ont rien à voir avec elle malgré la ressemblance des mots : le schéma `STAGING` de votre projet, qui est une couche de modélisation, et un *stage* Snowflake, qui est un emplacement de fichiers avant chargement. Trois notions, trois mots proches, aucun rapport.

Sous DuckDB, ni le transient ni la récupération après suppression n'existent : une table est une table, et le fichier de la base est votre seule sauvegarde.

Recoupements. Les vues appartiennent au Cours 03 de SQL, les expressions de table communes à son Cours 05 et le typage à son Cours 02. Les codes SAP que la `RAW` garde tels quels viennent du Cours 03 de Data Engineering, qui explique pourquoi il ne les décode pas : décoder à l'ingestion figerait le sens d'un code au jour du chargement.

## Points clés à retenir

- Un modèle dbt est un `SELECT` dans un fichier. dbt écrit le `create` autour, selon la matérialisation, et exécute dans l'ordre que le graphe dicte.
- Ce que dbt ajoute n'est pas de la puissance SQL : c'est un dépôt, un graphe, des tests et une documentation qui sortent du code.
- Le `dbt_project.yml` fait exister le projet ; le `profiles.yml` dit comment se connecter et vit hors du dépôt, avec un chemin absolu.
- Une source se déclare avant de se lire. `source()` fait remonter le graphe jusqu'à elle et met le nom du schéma à un seul endroit.
- Un schéma déclaré dans le projet s'ajoute à celui du profil au lieu de le remplacer : `generate_schema_name` rend les noms nus.
- Le `STAGING` renomme, type, décode et tire l'état courant. Il ne joint rien.
- L'état courant fait disparaître les doublons de livraison, et répare au passage l'histoire : c'est un choix, pas une évidence.
- Le staging se matérialise en vues ; les `MARTS` en tables.
- `ref()` relie les modèles et fonde le graphe. Nommer un modèle en dur fonctionne et casse tout ce que dbt sait faire.

---

## Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_03_AE.md`. Faites-les après le labo : plusieurs énoncés supposent que le projet tourne.



© 2026 iAlexMG. Tous droits réservés.